KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 1 ⋄ poznámky k přednášce

7. Rozšíření jazyka

verze z 6. listopadu 2024

V této části doplníme náš Lisp o několik operací, které budeme později potřebovat.

1 Speciální operátor `let*`

Speciální operátor `let*` vyžaduje argumenty ve stejném tvaru jako operátor `let` a pracuje velmi podobně.

```
                 popis prostředí
(let* ((a 2)  (b (+ a 2)) )
      popis vazby  popis vazby
  (* a b) )
   tělo
```

Popis prostředí Seznam libovolné délky. Jeho prvky jsou *popisy vazeb*.

Popis vazby Dvouprvkový seznam. Na prvním místě má symbol, na druhém libovolný výraz.

Tělo Libovolný výraz.

Vyhodnocení

Například pro výraz

```
(let* ((a 2)
       (b (+ a 2)))
  (* a b)))
```

Speciální operátor `let*` pracuje jako několik vnořených výrazů se speciálním operátorem `let`, každý vždy jen s jedním popisem vazby. Výraz nahoře je tedy ekvivalentní výrazu

```
(let ((a 2))
  (let ((b (+ a 2)))
    (* a b)))
```

2 Speciální operátor `quote`

Operátor přijímá jeden argument, jako výsledek tento argument vrací, aniž by ho vyhodnotil:

```
> (quote (1 2 3))
(1 2 3)

> (quote a)
A
```

Zkratka pro totéž je

```
> '(1 2 3)
(1 2 3)

> 'a
A
```

Měli bychom si ale stále pamatovat, že je to jen zkratka pro seznam se symbolem `quote` jako operátorem.

Symboly jsou tedy **hodnoty**. Příklady:

```
> (bind a 'b)
B

> a
B

> (bind b 'a)
A

> (list a b)
(B A)
```

```
> (bind c 'c)
C

> (eql c 'c)
T

> (eql 'c 'c)
T
```

Symbol jako argument i část návratové hodnoty funkce:

```
> (defun f (symbol)
    (list symbol))
F

> (f 'f)
(F)
```

Predikát `symbolp` testuje, zda daná hodnota je symbol (je to tedy typový predikát):

```
> (symbolp 'a)
T

> (symbolp nil)
T

> (symbolp 0)
NIL
```

Pomocí operátoru `quote` tedy **můžeme s každým symbolickým výrazem pracovat jako s hodnotou**:

```
> (car '(+ 1 2))
+

> (cdr '(+ 1 2))
(1 2)

> (car ''a)
QUOTE
```

K tomuto významnému faktu se ještě vrátíme.

Některé hodnoty ale nejsou správně utvořenými symbolickými výrazy: tečkové páry.

Pomocí operátoru `quote` lze ve zdrojovém kódu zadat výraz, jehož hodnotou je prázdný seznam takto:

```
'()
```

Jde samozřejmě o symbol `nil`, ale uvedený způsob zápisu zdůrazňuje, že na tomto místě ho chápeme jako prázdný seznam. Tento zápis prázdného seznamu preferujeme.

3 Hodnotová a funkční vazba

Už jsme si všimli, že názvy funkcí v Lispu nekolidují s názvy proměnných. Například tato funkce bude správně fungovat:

```
(defun list-test (list)
  (list list))
```

Test:

```
> (list-test (list 1 2))
((1 2))
```

Ve zdrojovém kódu je vždy poznat, zda symbol určuje funkci, nebo proměnnou: funkci určuje jedině, když je na prvním místě vyhodnocovaného seznamu (později se setkáme ještě s jedním případem).

Prakticky si můžeme představit, že existují **dva typy vazeb symbolů** tzv. *hodnotová* a *funkční*. Kdykoli jsme až doteď mluvili o vazbách symbolů, měli jsme na mysli hodnotovou vazbu. Je to vazba, jejíž hodnotou je hodnota proměnné, kterou symbol označuje (v aktuálním prostředí). Kromě toho může mít každý symbol i vazbu funkční, která určuje funkci, jíž je symbol názvem.

Prostředí, jak jsme je popisovali na začátku semestru, mají tedy složitější tvar. U každé vazby je kromě symbolu a jeho hodnoty ještě uvedeno, jakého je vazba typu.

Globální prostředí si můžeme představit takto:

Globální prostředí

symbol	**hodnota**	**typ vazby**
`pi`	`3.141592653589793D0`	*hodnotová*
`*`	*funkce* `*`	*funkční*
`car`	*funkce* `car`	*funkční*
`list`	*funkce* `list`	*funkční*
⋮	⋮	⋮

Během aplikace funkce `list-test` ukázané výše bude její prostředí takové:

Globální prostředí

symbol	hodnota	typ vazby
`list`	*funkce* `list`	*funkční*
⋮	⋮	⋮

Prostředí funkce `list-test`

symbol	hodnota	typ vazby
`list`	`(1 2)`	*hodnotová*

Hledání funkční vazby symbolu probíhá stejně jako hledání vazby hodnotové. Při vyhodnocování výrazu

```
(list list)
```

v těle funkce `list-test` se hledá jednak hodnotová, jednak funkční vazba symbolu `list`. Obě dvě se najdou. První rovnou v prostředí funkce `list-test` a druhá v jeho předkovi, tj. v globálním prostředí.

4 Speciální operátor `labels`

Už jsme se naučili vytvářet pomocí operátoru `let` nová prostředí s hodnotovými vazbami symbolů (není to ovšem jediný způsob, jak je vytvářet; další jsou operátor `let*` a aplikace uživatelsky definované funkce). K vytváření prostředí s funkčními vazbami symbolů slouží speciální operátor `labels`.

Operátor si nejprve vysvětlíme na příkladě. Na třetí přednášce jsme ukazovali iterativní verzi funkce na výpočet faktoriálu:

```
(defun fact-iter (n ir)
  (if (= n 0)
      ir
    (fact-iter (- n 1) (* ir n))))

(defun fact (n)
  (fact-iter n 1))
```

Takto by mohla vypadat její podoba s použitím operátoru `labels`:

```
(defun fact (n)
  (labels ((iter (n ir)
             (if (= n 0)
                 ir
               (iter (- n 1) (* ir n)))))
    (iter n 1)))
```

Operátor `labels` jsme použili k definování tzv. *lokální funkce* `iter`. Funkce není vidět nikde jinde než v těle funkce `fact`. Při vyhodnocování výrazu s operátorem `labels` totiž vzniká nové prostředí, ve kterém je funkční vazba symbolu `iter` (v našem případě) nastavena na novou funkci. Prostředí je aktuální pouze v těle operátoru `labels`, tedy pouze při vyhodnocování výrazu `(iter n 1)`.

Nová lokální funkce má tedy název `iter`. Její seznam parametrů je `(n ir)` a tělo je následující výraz s operátorem `if`.

Při vyhodnocování výrazu `(fact 10)`, v momentě, kdy je poprvé aplikována lokální funkce `iter`, vypadá aktuální prostředí takto:

Globální prostředí

symbol	**hodnota**	**typ vazby**
`fact`	*funkce* `fact`	*funkční*
⋮	⋮	⋮

↑

Prostředí funkce `fact`

symbol	**hodnota**	**typ vazby**
`n`	10	*hodnotová*

↑

Prostředí operátoru `labels`

symbol	**hodnota**	**typ vazby**
`iter`	*lokální funkce* `iter`	*funkční*

↑

Prostředí lokální funkce `iter`
(aktuální prostředí)

symbol	**hodnota**	**typ vazby**
`n`	10	*hodnotová*
`ir`	1	*hodnotová*

Vidíme, že předkem prostředí funkce `iter` není globální prostředí (jak je tomu u funkce definované operátorem `defun`), ale prostředí vytvořené operátorem `labels`.

Operátor `labels` může definovat několik nových lokálních funkcí současně. Jeho obecný tvar je tento:

```
(labels ((jméno1 parametry1 tělo1)
         (jméno2 parametry2 tělo2)
                     ⋮
         (jménon parametryn tělon))
  tělo)
```

Význam jednotlivých částí:

Jménoi Název i-té lokální funkce.

Parametryi Seznam parametrů i-té *lokální funkce*.

Těloi Tělo i-té lokální funkce. Tělo je vždy výraz.

Tělo Tělo samotného výrazu s operátorem `labels`. Opět je to výraz.

5 Nepovinné parametry funkce

Víme, že některé funkce (například funkci `+`, ale také mnoho dalších) můžeme aplikovat na libovolný počet argumentů. Ukážeme si, jak lze takové funkce definovat. Bude to jeden ze způsobů, jak zařídit, aby funkce měla tzv. nepovinné parametry. Další způsoby si ukážeme v příštím semestru.

Připomeňme si nejprve, jak jsme dosud používali makro `defun`:

```
(defun název-funkce parametry-funkce
  tělo-funkce)
```

Jak víme, *název-funkce* je symbol, *tělo-funkce* je výraz. *Parametry-funkce* je seznam symbolů:

```
(par1 par2 ... parn)
```

a říkáme jim *parametry funkce*. Seznam může mít (teoreticky) libovolnou délku, může být i prázdný (pro $n = 0$).

Takto definovanou funkci lze aplikovat na přesně tolik argumentů, kolik prvků má seznam parametrů. Proto také tyto parametry nazýváme *povinné parametry*.

Seznam parametrů funkce může také mít tento tvar:

```
(par1 par2 ... parn &rest rest-par)
```

V něm symbol `&rest` není parametrem funkce, ale určuje, že po něm v seznamu přijde jeden zvláštní parametr, kterému můžeme říkat *parametr pro zbylé argumenty*.

Funkci definovanou s takovým seznamem parametrů můžeme aplikovat na libovolný počet argumentů větší nebo roven n. Prvních n argumentů se jako obvykle naváže na povinné parametry *par1*, *par2* ... *parn*. Na parametr *rest-par* pro zbylé argumenty se pak naváže seznam všech dalších argumentů.

Z následujícího testu plyne, jak to funguje. Definujme funkci

```
(defun rest-test (a b &rest rest)
  (list a b rest))
```

a udělejme několik pokusů:

```
> (rest-test 'a 'b)
(A B NIL)

> (rest-test 'a 'b 'c)
(A B (C))

> (rest-test 'a 'b 'c 'd 'e 'f)
(A B (C D E F))

> (rest-test 'a)

Error: Function REST-TEST called with wrong number of arguments
(A).
```

Příklad (spojení libovolného počtu seznamů). Funkce `append` spojuje libovolný (i nulový) počet seznamů. Používá dvě lokální funkce (místo jedné z nich by šlo použít funkci `append-2`, kterou už máme; tu bychom teď mohli smazat).

```
(defun append (&rest lists)
  "Returns a list created by joining lists from the list LISTS."
  (labels ((app-2 (l1 l2)
             (if (null l1)
                 l2
               (cons (car l1) (app-2 (cdr l1) l2))))
           (app (lists)
             (if (null lists)
                 '()
               (app-2 (car lists) (app (cdr lists))))))
    (app lists)))
```

Nové symboly

speciální operátory: `let*`, `quote`, `labels`

funkce: `symbolp`

další: `&rest`

Otázky a úkoly na cvičení

Budou časem doplněny